

API接口文档——预制菜WMS仓储管理系统

1. 通用规范

1.1 请求规范

- **Content-Type:** `application/json;charset=utf-8`
- **认证:** 登录接口外, 所有接口需携带 `Authorization: Bearer <token>`
- **GET请求:** 自动添加 `_t` 时间戳防缓存(小程序端)

1.2 统一响应格式

```
{
  "code": 200,
  "msg": "操作成功",
  "data": { ... },
  "token": "eyJhbG..." // 仅登录接口返回
}
```

1.3 状态码

Code	含义
200	操作成功
500	操作失败
1001	用户名或密码错误
1002	令牌已过期或失效
1003	业务处理异常

1.4 认证白名单

以下路径不需要JWT令牌:

- `/auth/login` — 登录
- `/user` (POST) — 注册 (实际路径 `/user`, `JwtAuthenticationFilter` 通过 `@Component` 扫描放行)
- `/swagger-ui/**`, `/v3/api-docs/**` — API文档

2. 认证模块

2.1 AuthController — /auth

方法	路径	参数	说明
POST	/auth/login	Body: {username, password} (password 为AES-256-CBC加密后Base64)	用户登录, 返回用户信息+JWT令牌

请求示例:

```
POST /auth/login
{
  "username": "admin",
  "password": "Base64(AES加密密文)"
}
```

响应示例:

```
{
  "code": 200,
  "msg": "操作成功",
  "data": {
    "user_id": 1,
    "username": "admin",
    "real_name": "张三",
    "position": "主管",
    "department": "管理部门",
    "phone": "13800138000",
    "status": 1
  },
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
}
```

业务流程: AES解密 → BCrypt密码比对 → 生成JWT(2h有效, HMAC-SHA384) → 用户信息缓存 Redis(30min TTL)

3. 用户管理模块

3.1 UserController — /user

方法	路径	参数	说明
GET	/user/personnel	—	获取所有用户列表
GET	/user/{userId}	Path: userId	根据ID获取用户
POST	/user	Body: User对象	创建新用户 (密码BCrypt加密)
PUT	/user	Body: User对象	更新用户信息(密码AES解密后再BCrypt)
DELETE	/user/{id}	Path: id	删除用户
GET	/user/excel	—	导出用户列表Excel
PUT	/user/profile	Body: {user_id, old_password, new_password, real_name, phone, department, position}	个人资料修改(需验证旧密码)
GET	/user/inspectors	—	获取所有质检员 (position=质检员)
GET	/user/warehousestaff	—	获取所有仓库员工 (position=仓库员工)
GET	/user/checkers	—	获取盘点员 (position=仓库员工)

方法	路径	参数	说明
POST	<code>/user/migratePasswords</code>	—	批量迁移明文密码到BCrypt

4. 仓库管理模块

4.1 WarehouseController — `/warehouse`

方法	路径	参数	说明
GET	<code>/warehouse/list</code>	—	获取所有仓库
GET	<code>/warehouse/{warehouseId}</code>	Path: warehouseId	获取单个仓库
POST	<code>/warehouse</code>	Body: Warehouse对象	创建仓库(按size自动生成库区+库位)
PUT	<code>/warehouse</code>	Body: Warehouse对象	更新仓库(级联更新库区/库位名称)
DELETE	<code>/warehouse/{id}</code>	Path: id	删除仓库(仅空仓库可删,级联删除库区库位)
GET	<code>/warehouse/excel</code>	—	导出仓库Excel

POST `/warehouse` 自动生成规则:

- 大仓库: 16区 × 8库位 = 128库位
- 中仓库: 8区 × 8库位 = 64库位
- 小仓库: 4区 × 8库位 = 32库位

4.2 ZoneController — `/zone`

方法	路径	参数	说明
GET	<code>/zone/list</code>	Query: warehouseId(可选)	获取库区列表, 可按仓库筛选
GET	<code>/zone/{zoneId}</code>	Path: zoneId	获取单个库区
POST	<code>/zone</code>	Body: Zone对象	创建库区
PUT	<code>/zone</code>	Body: Zone对象	更新库区
DELETE	<code>/zone/{id}</code>	Path: id	删除库区

4.3 LocationController — /location

方法	路径	参数	说明
GET	/location/list	Query: zoneId(可选)	获取库位列表，可按库区筛选
GET	/location/{locationId}	Path: locationId	获取单个库位
POST	/location	Body: Location对象	创建库位
PUT	/location	Body: Location对象	更新库位
DELETE	/location/{id}	Path: id	删除库位

5. 货物与批次模块

5.1 GoodsController — /goods

方法	路径	参数	说明
GET	/goods/list	—	获取所有货物
GET	/goods/{goodsId}	Path: goodsId	获取单个货物
POST	/goods	Body: Goods对象	创建货物(自动分配最小可用ID, quantity=0)
PUT	/goods	Body: Goods对象	更新货物
DELETE	/goods/{id}	Path: id	删除货物(仅quantity=0时可删)
GET	/goods/excel	—	导出货物Excel

5.2 BatchController — /batch

方法	路径	参数	说明
GET	/batch/list	—	获取所有批次
GET	/batch/{batchId}	Path: batchId	获取单个批次
POST	/batch	Body: Batch 对象	创建批次(自动生成编号B+日期+流水, 创建待入库库存)
PUT	/batch	Body: Batch 对象	更新批次
DELETE	/batch/{id}	Path: id	删除批次
GET	/batch/byStatus	Query: status	按状态筛选批次
GET	/batch/excel	—	导出批次Excel

6. 库存模块

6.1 InventoryController — /inventory

方法	路径	参数	说明
GET	/inventory/list	—	获取所有库存
GET	/inventory/listwithDetails	Query: batchId(可选)	获取库存详情(关联货物名、库位名、仓库名)
GET	/inventory/{id}	Path: id	获取单个库存记录
POST	/inventory	Body: Inventory对象	创建库存记录
PUT	/inventory	Body: Inventory对象	更新库存记录
DELETE	/inventory/{id}	Path: id	删除库存记录
GET	/inventory/excel	—	导出库存Excel(含详情)

7. 入库业务模块 (核心)

7.1 InboundController — /inbound

方法	路径	参数	说明
GET	/inbound/availableBatches	—	获取可入库批次 (待入库且无活跃任务)
POST	/inbound/create	Body: CreateInboundRequest	创建入库任务: 生成入库单(↑+日期+流水), 创建单据+任务+日志
PUT	/inbound/reject/{taskId}	Path: taskId, Body: {operatorId}	拒绝入库: 删除入库单和任务
PUT	/inbound/complete/{taskId}	Path: taskId, Body: CompleteInboundRequest	执行入库: 分配库位, 创建库存, 标记库位占用, 更新区/仓可用数, 更新货物数量, 批次→正常

POST /inbound/create 请求体:

```
{
  "inboundType": "采购入库",
  "batchId": "B20260711001",
  "operatorId": 3,
  "priority": "高",
  "deadline": "2026-07-20",
  "remark": "紧急入库"
}
```

PUT /inbound/complete/{taskId} 请求体:

```
{
  "warehouseId": 1,
  "locationIds": [1, 2, 3, 5],
  "remark": "已入库",
  "operatorId": 3
}
```

complete业务逻辑 (10张表联动):

1. 验证库位数量 = ceil(初始数量/10)
2. 验证库位未锁定
3. 创建inventory记录(状态→正常)
4. 标记location.is_occupied=1
5. zone.available_slots -= 占用数
6. warehouse.available_slots -= 占用数
7. goods.quantity += 入库数量

- 8. batch.status → '正常'
- 9. inbound_order_head.status → '已完成'
- 10. 记录operation_log

7.2 入库单CRUD

InboundOrderHeadController — /inboundOrderHead

方法	路径	说明
GET	/inboundOrderHead/list	列表
GET	/inboundOrderHead/{orderNo}	详情
POST	/inboundOrderHead	创建
PUT	/inboundOrderHead	更新
DELETE	/inboundOrderHead/{orderNo}	删除

InboundOrderDetailController — /inboundOrderDetail

方法	路径	说明
GET	/inboundOrderDetail/list	列表
GET	/inboundOrderDetail/{id}	详情
POST	/inboundOrderDetail	创建
PUT	/inboundOrderDetail	更新
DELETE	/inboundOrderDetail/{id}	删除

8. 出库业务模块 (核心)

8.1 OutboundController — /outbound

方法	路径	参数	说明
POST	<code>/outbound/create</code>	Body: CreateOutboundRequest	创建出库任务: 生成出库单 (O+日期+流水), 锁定源库位, 锁定库存
PUT	<code>/outbound/reject/{taskId}</code>	Path: taskId, Body: {operatorId}	拒绝出库: 解锁库位, 恢复锁定库存
PUT	<code>/outbound/complete/{taskId}</code>	Path: taskId, Body: {operatorId}	执行出库: 扣减库存, 释放空库位, 更新货物数量, 更新批次剩余量
GET	<code>/outbound/detail/{taskId}</code>	Path: taskId	获取出库任务详情(含货物名/库位名)
GET	<code>/outbound/availableInventory</code>	Query: goodsId	获取可用库存(按FIFO排序)

POST /outbound/create 请求体:

```
{
  "items": [{
    "goodsId": 1,
    "batchId": "B20260711001",
    "quantity": 20,
    "warehouseId": 1,
    "zoneId": 1,
    "locationId": 3
  }],
  "assigneeId": 3,
  "priority": "高",
  "deadline": "2026-07-20",
  "operatorId": 1
}
```

complete业务逻辑:

- 扣减inventory.quantity和locked_quantity
- quantity归零→释放location(加zone/warehouse可用数)
- goods.quantity -= 出库数量
- batch.remaining_quantity -= 出库数量
- batch.remaining归零→级联删除(batch→QC详情→盘点详情→出库详情→库存)

8.2 出库单CRUD

OutboundOrderHeadController — /outboundOrderHead

方法	路径	说明
GET	/outboundOrderHead/list	列表
GET	/outboundOrderHead/{orderNo}	详情
POST	/outboundOrderHead	创建
PUT	/outboundOrderHead	更新
DELETE	/outboundOrderHead/{orderNo}	删除

OutboundOrderDetailController — /outboundOrderDetail

方法	路径	说明
GET	/outboundOrderDetail/list	列表
GET	/outboundOrderDetail/{id}	详情
POST	/outboundOrderDetail	创建
PUT	/outboundOrderDetail	更新
DELETE	/outboundOrderDetail/{id}	删除

9. 质检业务模块 (核心)

9.1 QualityCheckController — /qualityCheck

方法	路径	参数	说明
POST	/qualityCheck/create	Body: CreateQcRequest	创建质检任务: 生成质检单(C+日期+流水), 验证检查类型与库存状态匹配
PUT	/qualityCheck/complete/{taskId}	Path: taskId, Body: CompleteQcRequest	完成质检: 记录结果, 合格→批次正常, 不合格→批次报废+库存待报废
GET	/qualityCheck/myTasks	Query: assigneeId	获取用户的质检任务

POST /qualityCheck/create 请求体:

```
{
  "checkType": "来料检",
  "batchId": "B20260711001",
  "inspectorId": 5,
  "priority": "中",
  "deadline": "2026-07-21",
  "remark": "",
  "operatorId": 1
}
```

PUT /qualityCheck/complete/{taskId} 请求体:

```
{
  "inspectionResult": "不合格",
  "defectReason": "菌落超标",
  "handlingSuggestion": "报废处理",
  "remark": "",
  "operatorId": 5
}
```

质检类型与库存状态校验:

- 来料检 → 库存状态必须为"待入库"
- 日常抽检 → 库存状态必须为"正常"

9.2 质检单CRUD

QualityCheckHeadController — /qualityCheckHead (5个CRUD端点)

QualityCheckDetailController — /qualityCheckDetail (5个CRUD端点)

10. 库存盘点业务模块 (核心)

10.1 InventoryCheckController — /inventoryCheck

方法	路径	参数	说明
POST	<code>/inventoryCheck/create</code>	Body: CreateInventoryCheckRequest	创建盘点任务: 生成盘点单(P+日期+流水), 按范围生成盘点明细
PUT	<code>/inventoryCheck/complete/{taskId}</code>	Path: taskId, Body: CompleteInventoryCheckRequest	完成盘点: 录入实盘数, 计算差异, 更新库存, 重算区/仓可用数
GET	<code>/inventoryCheck/myTasks</code>	Query: assigneeId	获取用户的盘点任务
GET	<code>/inventoryCheck/details</code>	Query: checkNo	获取盘点明细

POST /inventoryCheck/create 请求体:

```
{
  "checkType": "按库位盘点",
  "scopeType": "location",
  "scopeValue": "3",
  "operatorId": 1,
  "priority": "中",
  "deadline": "2026-07-22",
  "remark": ""
}
```

盘点范围类型: location(按库位), warehouse(按仓库), goods(按货物), batch(按批次)

complete业务逻辑:

1. 更新每个detail的actual_quantity
2. 计算difference = actual - book
3. 差异=0→正常, >0→盘盈, <0→盘亏
4. 更新inventory.quantity为实盘数
5. inventory归零→释放location
6. 重算zone.available_slots, warehouse.available_slots
7. 重算batch.remaining_quantity
8. 重算goods.quantity

10.2 盘点单CRUD

InventoryCheckHeadController — `/inventoryCheckHead` (5个CRUD端点)

11. 库存调整模块 (核心)

11.1 AdjustmentController — /adjustment

方法	路径	参数	说明
POST	/adjustment/create	Body: CreateAdjustmentRequest	创建调整任务: 生成调整单(A+日期+流水), 锁定源+目标库位
PUT	/adjustment/reject/{taskId}	Path: taskId, Body: {operatorId}	拒绝调整: 解锁库位
PUT	/adjustment/complete/{taskId}	Path: taskId, Body: {operatorId}	完成调整: 移动库存(更新 warehouse/zone/location), 释放源库位, 占用目标库位
GET	/adjustment/detail/{taskId}	Path: taskId	获取调整详情(含货物名,源/目标库位名)

POST请求体items每项:

```
{
  "goodsId": 1, "batchId": "B20260711001", "quantity": 10,
  "fromWarehouseId": 1, "fromZoneId": 1, "fromLocationId": 3,
  "towarehouseId": 1, "toZoneId": 2, "toLocationId": 12
}
```

11.2 调整单CRUD

AdjustmentOrderHeadController — /adjustmentOrderHead (5个CRUD端点)

AdjustmentOrderDetailController — /adjustmentOrderDetail (5个CRUD端点)

12. 次品处理模块 (核心)

12.1 DefectiveController — /defective

方法	路径	参数	说明
POST	/defective/create	Body: CreateDefectiveRequest	创建次品处理任务: 生成处理单 (D+日期+流水), 锁定源库位
PUT	/defective/reject/{taskId}	Path: taskId, Body: {operatorId}	拒绝: 解锁库位
			完成: 删除库存,

方法	/defective/complete/{taskId}	Path: taskId, Body: 参数 {operatorId}	说明 释放库位, 减少货物数量, 级联删除批次及相关记录
GET	/defective/detail/{taskId}	Path: taskId	获取处理详情

complete级联删除链: batch → QC详情 → 盘点详情 → 出库详情 → inventory → batch表

12.2 次品处理单CRUD

DefectiveHandlingHeadController — /defectiveHandlingHead (5个CRUD端点)

DefectiveHandlingDetailController — /defectiveHandlingDetail (5个CRUD端点)

13. 任务与日志模块

13.1 WorkTaskController — /workTask

方法	路径	参数	说明
GET	/workTask/list	—	获取所有任务
GET	/workTask/{id}	Path: id	获取单个任务
POST	/workTask	Body: WorkTask	创建任务
PUT	/workTask	Body: WorkTask	更新任务
DELETE	/workTask/{id}	Path: id	删除任务
GET	/workTask/myTasks	Query: assigneeld	获取指定用户的未完成任务(按优先级+截止日期排序)

13.2 OperationLogController — /operationLog

方法	路径	参数	说明
GET	/operationLog/list	Query: date(可选), operationType(可选)	获取操作日志(支持按日期+类型筛选)
GET	/operationLog/{id}	Path: id	获取单条日志
POST	/operationLog	Body: OperationLog	创建日志
PUT	/operationLog	Body: OperationLog	更新日志
DELETE	/operationLog/{id}	Path: id	删除日志
GET	/operationLog/excel	—	导出日志Excel

14. AI与数据看板模块

14.1 AiController — /ai

方法	路径	参数	说明
POST	/ai/chat	Body: {question, history[]}	AI对话: 发送问题+历史对话, 返回DeepSeek回复
GET	/ai/homeStats	Query: assigneeId(可选)	首页统计: 总库存(排除待入库/待报废), 待办任务数, 最后更新时间

POST /ai/chat 请求体:

```
{
  "question": "冷冻鸡排还有多少库存?",
  "history": [
    {"role": "user", "content": "今天入库了多少?"},
    {"role": "assistant", "content": "今日入库..."}
  ]
}
```

AI处理流程:

- 1. 关键词匹配 → 查询真实数据库
- 2. 组装: System Prompt + Data Context + History + Question
- 3. OkHttp → DeepSeek API
- 4. 返回AI回复

14.2 DashboardController — /dashboard

方法	路径	说明
GET	/dashboard/weeklyInboundOutbound	近7天出入库趋势: {dates[], inbound[], outbound[]}
GET	/dashboard/goodsQuantity	货物数量分布: {names[], quantities[]}
GET	/dashboard/warehouseTempHumidity	仓库温湿度: {names[], temperatures[], humidities[]}
GET	/dashboard/warehouseCapacity	仓库容量: [{warehouseName, totalSlots, usedSlots, availableSlots, saturation}]

15. 温湿度模块

15.1 TemperatureHumidityRecordController — /temperatureHumidityRecord

方法	路径	参数	说明
GET	/temperatureHumidityRecord/list	—	获取所有温湿度记录
GET	/temperatureHumidityRecord/{id}	Path: id	获取单条记录
POST	/temperatureHumidityRecord	Body: 记录对象	创建记录
PUT	/temperatureHumidityRecord	Body: 记录对象	更新记录
DELETE	/temperatureHumidityRecord/{id}	Path: id	删除记录
POST	/temperatureHumidityRecord/upload	Body: {warehouseId, temperature, humidity}	上传温湿度: upsert逻辑(有则更新,无则创建)

16. 接口调用统计

控制器	路径前缀	接口数	核心业务
HelloController	/h	1	健康检查
AuthController	/auth	1	AES+BCrypt登录
UserController	/user	11	用户CRUD+角色列表
WarehouseController	/warehouse	6	仓库CRUD+Excel+自动生成
ZoneController	/zone	5	库区CRUD
LocationController	/location	5	库位CRUD
GoodsController	/goods	6	货物CRUD+Excel
BatchController	/batch	7	批次CRUD+状态筛选
InventoryController	/inventory	7	库存CRUD+详情+Excel
InboundController	/inbound	4	入库流程 (create/reject/complete)
InboundOrderHeadController	/inboundOrderHead	5	入库单头CRUD
InboundOrderDetailController	/inboundOrderDetail	5	入库单明细CRUD
OutboundController	/outbound	5	出库流程 +可用库存
OutboundOrderHeadController	/outboundOrderHead	5	出库单头CRUD
OutboundOrderDetailController	/outboundOrderDetail	5	出库单明细CRUD
QualityCheckController	/qualityCheck	3	质检流程 +我的任务
QualityCheckHeadController	/qualityCheckHead	5	质检单头CRUD
QualityCheckDetailController	/qualityCheckDetail	5	质检单明细CRUD
InventoryCheckController	/inventoryCheck	4	盘点流程 +明细
InventoryCheckHeadController	/inventoryCheckHead	5	盘点单头CRUD
InventoryCheckDetailController	/inventoryCheckDetail	5	盘点单明细CRUD
AdjustmentController	/adjustment	4	移库流程 +详情
AdjustmentOrderHeadController	/adjustmentOrderHead	5	调整单头CRUD
AdjustmentOrderDetailController	/adjustmentOrderDetail	5	调整单明细CRUD
DefectiveController	/defective	4	次品处理流程 +详情
DefectiveHandlingHeadController	/defectiveHandlingHead	5	次品处理头CRUD
DefectiveHandlingDetailController	/defectiveHandlingDetail	5	次品处理明细CRUD
WorkTaskController	/workTask	6	任务CRUD+我的任务
OperationLogController	/operationLog	6	日志CRUD+筛选+Excel
AIController	/ai	2	AI聊天+首页统计
DashboardController	/dashboard	4	看板4图表数据
TemperatureHumidityRecordController	/temperatureHumidityRecord	6	温湿度CRUD+上传
合计		142	

